CSC3100 Assignment2 Solution

1 Q1

The problem can be done recursively according to the steps described in the question. For any number n, first, use a stack to store the number of powers of binary decomposition, then the number at the top of the stack is the largest, for example, $137 = 2^7 + 2^3 + 1$ then the stack is [0, 3, 7] from the bottom up. After binary decomposition, keep popping the largest number from the stack and outputting it, if the element at the top of the stack is 0 or 1, output it directly. If the element at the top of the stack is not 0 or 1, recursively decompose and output them.

2 Q2

For the given n wires, it contains $\frac{n(n-1)}{2}$ crossings at most.
Now look at two wires $i->a_i$ $j->a_j$
If $a_i > a_j$ then the wire i must cross with wire j.
If $a_i < a_j$ then the wire i must not cross with wire j.
Consider what will happen if $a_i = a_j$.
Since we want to achieve the maximum number of crossings.
We place the front one on the upper terminal at the back of the lower terminal.
Then wire i and wire j must form a crossing.
The answer is $\sum_{i=1}^{n} \sum_{j=i+1}^{n} [a_i \geq a_j]$, where $[P] = 1$ if P is true, else $[P] = 0$
It can be solved by merge sort in $O(nlogn)$

3 Q3

This topic of Q3 is about writing basic recursive functions. First create a new $3^k \times 3^k$ two-dimensional char array, recursively divide the whole two-dimensional array into $3 \times 3 = 9$ parts each time, modify all the middle subsquare to ' ' characters, and do the same operation for the other 8 subsuqare. According to Master Theorem, the time complexity is $T(n) = 8 * T(n/3) + \Theta(n^2) = \Theta(n^2)$, where $n = 3^k$

4 Q4

Build a bst to maintain the array.
For each node of the bst, the key is the element's position in the array, the value is the element's value.
In order to maintain the key of each node efficiently, we define $size_u = \sum_{v \in subtree(u)} 1$.
When inserting or deleting a node, we can get the key of each node by finding the size value of its left child and the remains of the previous node. Then we can insert or delete a node as we do in a common bst.
After inserting a node or deleting a node, there will be at most $O(h)$ node's size changed, where h is the height of the bst.
Then we can solve the insert and delete operation in $O(h)$.
Still, we can maintain the sum as we do to maintain size. we define $sum_u = \sum_{v \in subtree(u)} value_v$.
If we need to calculate $sum(l, r)$, it can be reformulated to $sum(1, r) - sum(1, l-1)$.
We can caculate $sum(1, x)$ in $O(h)$ since we have maintained sum for each node.
Details for this problem are shown in the code.